

Notes 9

1. grep

Definition

grep stands for **Global Regular Expression Print**. It searches for specific patterns (text or regular expressions) in one or more files or input streams and prints the lines that match the pattern.

Usage / Formula

```
grep [OPTIONS] PATTERN [FILE...]
```

Common Options:

- **-i** → case-insensitive search
- **-r** or **-R** → recursive search through directories
- **-v** → invert match (show non-matching lines)
- **-c** → count the number of matching lines
- **-n** → show line numbers
- **-E** → use extended regular expressions
- **-l** → show only filenames that contain matches

Examples

```
grep "error" app.log
```

```
grep -ri "TODO" ./project/
```

```
grep -n "function" script.sh
```

```
grep -c "success" results.txt
```

```
grep "localhost" /etc/hosts /etc/hostname
```

```
ps aux | grep "nginx"
```



GREP



- **Description:**
 - Grep is used to search text in a given file(s). Grep works line by line. (it matches the search criteria in a line by line basis).
- **Usage:**
 - `grep + option + search criteria + file(s)`
- **Basic Example:**
 - Search any line that contains the word "dracula" in the given file:
 - `grep 'dracula' ~/Documents/dracula.txt`
 - Search any line that contains the word 'dracula' regardless of the case
 - `grep -i 'dracula' ~/Documents/Books/dracula.txt`
 - Display how many lines contain the matched string
 - `grep -c 'dracula' ~/Documents/Books/dracula.txt`



Common options of grep



Option	Explanation
-i	Enables case insensitivity. (it will match regardless of case)
-n	Displays line number for every line matched
-E	Treats the pattern (search criteria) as an extended regular expression
-G	Treats the pattern (search criteria) as a basic regular expression
-v	Inverts the search (looks for the opposite of the given criteria)
-o	Only displays the matched string
-c	Search and display the total number of times a pattern is matched
-w	Matches only the given word (pattern) by itself.
-r, -R	Matches recursively. Useful for searching files in a given directory

6

2. awk

Definition

awk is a powerful text-processing and data extraction tool. It works excellently with column-based (structured) text, allowing you to filter, calculate, and reformat data easily.

Usage / Formula

```
awk 'PATTERN { ACTION }' [FILE...]
```

Key Variables:

- `$1, $2, ...` → fields (columns)
- `$0` → entire line
- `NF` → number of fields in current line
- `NR` → current line number
- `BEGIN {}` → runs before processing
- `END {}` → runs after processing

Examples

```
awk '{print $1, $3}' data.txt
```

```
awk '$3 > 100 {print $0}' sales.csv
```

```
awk '{sum += $2} END {print "Total:", sum}' numbers.txt
```

```
awk -F ',' '{print $1 " has score: " $2}' scores.csv
```

```
ls -l | awk '{print $9}'
```

```
awk 'BEGIN {print "Name\tScore"} {print $1 "\t" $2} END {print "Done"}' data.txt
```



The awk command



- **Description:**
 - Awk is a scripting language used for processing and displaying text. Awk can work with a text file or from standard output. Awk was created in Bell Labs during the 70s by Alfred Aho, Peter Weinberger, and Brian Kernighan and its name comes from its authors' initials. There are several implementations of Awk: nawk, mawk, gawk, and busybox.
 - Awk performs operations line by line
- **Usage:**
 - `awk + options + {awk command} + file + file to save (optional)`
- **Basic Example:**
 - Print the first column of every line of a file
 - `awk '{print $1}' ~/Documents/Csv/cars.csv`

1
3

Awk Variables



\$0	Whole line
\$1, \$2... \$NF	First, second... last field
NR	Total Number of Records
NF	N number of Fields
OFS	Output Field Separator (default " ")
FS	input Field Separator (default " ")
ORS	Output Record Separator (default "\n")
RS	input Record Separator (default "\n")
FILENAME	Name of the file
ARGC	Number of arguments

ARGV	Array of arguments
FNR	File Number of Records
OFMT	Format for numbers (default "%.6g")
RSTART	Location in the string
RLENGTH	Length of match
SUBSEP	Multi-dimensional array separator (default "\034")
ARGIND	Argument Index
ENVIRON	Environment variables
IGNORECASE	Ignore case
CONVFMT	Conversion format
ERRNO	System errors
FIELDWIDTHS	Fixed width fields

1
4

3. sed

Definition

sed (Stream Editor) is a tool for filtering and transforming text. It is most commonly used for find-and-replace operations, deletions, and text substitutions using regular expressions.

Usage / Formula

`sed [OPTIONS] 'SCRIPT' [FILE...]`

Common Commands:

- `s/old/new/` → substitute (replace)
- `s/old/new/g` → global replace on line
- `d` → delete line
- `-i` → edit file in place

Examples

```
sed 's/error/warning/' app.log
```

```
sed 's/foo/bar/g' file.txt
```

```
sed 's/old/new/3' file.txt
```

```
sed '/debug/d' app.log
```

```
sed -i.bak 's/localhost/127.0.0.1/g' config.txt
```

```
cat file.txt | sed 's/[0-9]/X/g'
```



The sed command



- **Description**

- SED is a stream editor that performs operations on files and standard output. For instance it can search, find and replace, insert, and delete. By using SED you can edit files without opening them

- **Usage**

- `sed + options + sed script + file`

- **Basic Example**

- Replacing a string in given file globally (replace false for true)

- `sed 's/false/true/g' ~/Documents/sample_files/Json/joke.json`

2
1

More examples of sed



- Replacing only the fourth occurrence per line in a file

- `sed 's/false/true/4' joke.json`

- Replacing from the given number occurrence to the rest occurrences in a file. Start at the second time the word appears and continue to till the end of the file

- `sed 's/false/true/3g' joke.json`

- Replacing string on a specific line number

- `sed '3 s/false/true/' joke.json`

- Replacing string on a range of lines

- `sed '1,3 s/false/true/' joke.json`

2
2